CREST-GV vs CHEERS: Pipeline, RPKM Justification, and Results

CHEERS repository - CREST-GV_vs_CHEERS pipeline. 2026-10-15.

1. Purpose

CREST-GV produces, for each of 12 reference chromatin accessibility collections, a per-cell-type enrichment statistic describing how strongly a set of trait-associated SNPs falls into that cell type's accessible chromatin. This pipeline runs the independent, previously published CHEERS (Chromatin Element Enrichment Ranking by Specificity) method on the same underlying data - RPKM-normalized bigWig tracks and per-cell-type peak calls - so that CHEERS's enrichment p-values can be compared against CREST-GV's own statistic, collection by collection and trait by trait.

CHEERS was not designed to consume bigWig tracks; it expects raw integer read counts per peak. Section 3 explains why RPKM-normalized bigWig signal can be used directly, without reconstructing raw counts, and why that is not a compromise but a mathematically exact substitution given CHEERS's own normalization steps.

2. Input data

2.1 CREST-GV collections

Location: /project/Wellcome_Discovery/datashare/CREST-GV_collection/

Twelve collections are used, each with an identical internal layout:

Internal key	On-disk directory
cad	CAD
calderon	calderon
catlas_fetal	catlas_fetal
catlas_adult	catlas_adult
erythoid_d7_d10_d13_d17	Days7_10_13_17
h1_hescs	H1_hESCs
immune_cell	immune_cell
ludwig2019	ludwig2019
mpal	MPAL
pancreatic_pbmc	pancreatic_pbmc
super_pbmc	super_PBMC
greenleaf_brain	greenleaf_brain

Within each collection directory:

```
<collection>/
  bigwigs/<cell_type>.bw        # RPKM-normalized, bamCoverage --binSize 1
  peaks/<cell_type>_L-tron.bed  # per-cell-type peak calls (chr, start, end)
  <collection>_info.csv         # cell counts per type
  <collection>_qc.csv           # QC labels per type
  <collection>_ps.csv           # CREST-GV background calibration (n_snps, p_success)
```

Peak calls are made independently per cell type - there is no shared consensus peak set across cell types within a collection, which CHEERS requires (its normalization inner-joins all samples on identical chr/start/end coordinates).

2.2 Trait SNP lists

Four GWAS traits, already in the exact format CHEERS expects (name, chr, position, tab-

separated, no header). All four live under /project/Wellcome_Discovery/shared/imputed_genetics_per_trait_collection/, one file per trait:

Trait	Filename
type_1_diabetes_mellitus	type_1_diabetes_mellitus.bed
type_2_diabetes_mellitus	type_2_diabetes_mellitus.bed
red_blood_cell_traits	red_blood_cell_traits.bed
heart_rate	heart_rate.bed

3. Why RPKM bigWig signal can be used directly (no reversal needed)

3.1 The problem

The bigWigs were generated with bamCoverage --normalizeUsing RPKM --binSize 1. The original BAM files and per-sample mapped-read counts are not available. RPKM normalization applies, per sample, a single scale factor:

scale_i = 1e9 / mapped_reads_i (at binSize = 1 bp)

Because RPKM is defined specifically to divide out each sample's total mapped-read count, the genome-wide sum of an RPKM track is INVARIANT to that count. mapped_reads_i cannot be recovered from the bigWig alone - true reversal of RPKM is mathematically impossible, not merely inconvenient.

3.2 Why this does not matter for CHEERS

CHEERS_normalize.py applies four steps to a peak x sample count matrix. Feeding it RPKM-summed values instead of true raw counts: (1) normalize_counts() rescales every sample to match the largest total signal - substituting the RPKM scale factor shows the result differs from the raw-count result by exactly ONE GLOBAL CONSTANT across every sample and peak, not a per-sample bias; (2) drop_peaks() (10th-percentile filter) is invariant to any positive uniform rescaling; (3) quantile_norm() operates on within-column ranks, unaffected by positive scaling; (4) euclidean_norm() divides each peak's row by its own Euclidean norm, which EXACTLY CANCELS any remaining global constant: (k*z)/||k*z|| = z/||z||.

CONCLUSION: the final specificity scores are mathematically identical whether CHEERS is given true raw counts or RPKM-normalized bigWig signal over the same peaks, provided the RPKM scale factor is a pure per-sample constant applied uniformly - exactly what bamCoverage --normalizeUsing RPKM --binSize 1 produces. No RPKM reversal is performed anywhere in this pipeline.

- Holds exactly only with consistent bamCoverage settings across all bigWigs in a collection (same --binSize, no sample-specific --ignoreForNormalization).
- Raw integer counts can tie exactly; RPKM floats essentially never do - a negligible tie-breaking difference in rank(method='first'), not a structural one.

4. Pipeline implementation

```
CREST-GV_vs_CHEERS/
  config.py                    # paths, collection map, trait map, tool paths
  scripts/
    build_consensus_peaks.py   # step A, per collection
    quantify_bigwigs.py        # step B, per collection
    run_collection.py          # orchestrates A -> D for one collection
    compare_with_crestgv.py    # joins CHEERS output vs CREST-GV per cell type
```

data/	*consensus_peaks/, counts/<collection>/<cell_type>_ReadsInPeaks.txt*
results/	*normalize/<collection>/..., enrichment/<collection>/<trait>/..., comparison/<collection>__<trait>.tsv, summary.tsv*
docs/	*this report, interactive scatter-plot HTML*

Step A (build_consensus_peaks.py) merges every cell type's peak BED file within a collection via bedtools sort + merge into one non-overlapping consensus set, with a unique peak ID added for bigWigAverageOverBed. Step B (quantify_bigwigs.py) runs bigWigAverageOverBed for every cell type's bigWig against the consensus peaks and takes the sum column (total RPKM signal over the peak). Steps C/D run the repository's existing, unmodified CHEERS_normalize.py and CHEERS_computeEnrichment.py.

- awk and GNU parallel are not installed on this cluster; all text processing is done in Python.
- bedtools and bigWigAverageOverBed are only available via module load; since Lmod's module command is a shell function (not visible to subprocess.run), config.py hardcodes resolved absolute binary paths instead.
- The system default python3 has a broken NumPy (missing libblas.so.3); config.py resolves the python-cbrg module's interpreter plus its LD_LIBRARY_PATH and PYTHONPATH explicitly for subprocess calls.

5. Results

The pipeline was run to completion on all 12 collections across all 4 traits (48 collection/trait pairs, ~523 bigWigs). All 48 pairs produced normalized specificity scores and enrichment p-values; none failed.

5.1 A real bug found and fixed during QA: corrupted bigWigAverageOverBed output

Every collection's raw quantified counts were scanned for implausible values before trusting the results. This caught a genuine, reproducible bug: for greenleaf_brain, bigWigAverageOverBed (ucsctools v385) returned a garbage sum value (1.8783e+300, or -5.8e+305 in one file) for one specific peak (chr1:248944146-248946588) in 5 of its 20 bigWigs, instead of erroring.

That single corrupted value silently invalidated the ENTIRE collection's result: normalize_counts()'s library-size scaling and quantile_norm()'s shared rank-value table both pool across every peak and sample, so the contamination spread to all 20 cell types; euclidean_norm() then squared those now-astronomical values, overflowing to infinity, and finite/infinity = 0.0 silently collapsed the entire 257,279-peak x 20-cell-type specificity matrix to zero - no exception was ever raised. Every peak tied for rank 1, and every enrichment p-value for greenleaf_brain was degenerate, for all 4 traits.

Fix: quantify_bigwigs.py now rejects any bigWigAverageOverBed output that is negative, non-finite, or exceeds a sanity ceiling (1e9, far above the ~1e7 maximum legitimate value observed across all 12 collections), replacing it with 0.0 and logging a warning. All other 11 collections were re-scanned for the same signature and found clean; only greenleaf_brain was affected. After the fix, its specificity matrix shows 0% degenerate rows and sensible enrichment p-values.

5.2 Overall concordance with CREST-GV

CHEERS's output was compared against CREST-GV's own CREST-GV_pval_norm column, not the raw CREST-GV_pval, via Spearman rank correlation (rho), per collection/trait pair, across 48 pairs. CREST-GV computes CREST-GV_pval_norm as norm.sf(Z_norm), where Z_norm=(Z-mean(Z))/std(Z) is re-standardized across all cell types WITHIN one collection

(crestgv.py, __add_statistics()). This re-standardization is what makes the statistic comparable across collections of very different size and composition - the raw CREST-GV_pval is calibrated only against a trait-independent, genome-wide background rate and does not correct for collection-specific scale differences, so CREST-GV_pval_norm is the column its own implementation intends for this kind of comparison. Median rho = 0.594, mean rho = 0.552.

Trait	Median rho	Range
Type 1 diabetes	0.706	0.343 - 0.933
Red blood cell traits	0.594	-0.143 - 1.000
Heart rate	0.513	-0.190 - 0.907
Type 2 diabetes	0.502	0.036 - 1.000

Collection	Median rho	Collection	Median rho
catlas_adult	0.785	super_pbmc	0.444
cad	0.761	erythoid_d7_d10_d13_d17	0.420
greenleaf_brain	0.751	pancreatic_pbmc	0.358
catlas_fetal	0.746	calderon	0.122
h1_hescs	0.700	ludwig2019	0.024
immune_cell	0.567	mpal	0.525

Caveat on raw significance counts: counting cell types 'significant' at p<0.05 is lopsided and should NOT be read as a concordance metric - CHEERS flags 1,314 cell types significant across all pairs, CREST-GV's normalized statistic flags only 78 (71 overlapping). This is not CHEERS finding more real signal; its raw p-value has no cross-collection calibration and underflows toward 0 very easily in large, strongly-enriched collections (Section 5.4). Rank correlation (rho), not the significance count, is the concordance metric used throughout.

5.3 Biological validation: top enriched cell types per trait

As an independent face-validity check, CHEERS's own top hits per trait, ranked by Z-score across all 12 collections, recover the expected biology in every case:

- Type 1 diabetes: T lymphocytes (CD4+/CD8+), thymocytes, cytotoxic T, macrophages - consistent with T1D's T-cell-mediated autoimmune pathology.
- Type 2 diabetes: pancreatic acinar, hepatoblast, skeletal myocytes (type I/II), adipocyte - metabolic/insulin-resistance tissues, not immune cells.
- Red blood cell traits: erythroblasts (stages 1-4), megakaryocyte - directly the erythroid lineage (Z up to 73, the strongest signal of all 4 traits).
- Heart rate: atrial and ventricular cardiomyocytes, vascular smooth muscle, endothelial - cardiac tissue, as expected.

5.4 A statistical caveat for small collections

Four collections have few enough cell types that their individual Spearman rho should not be over-interpreted: h1_hescs (n=4), ludwig2019 (n=8), CAD (n=10), pancreatic_pbmc (n=12). Checking significance of the correlation itself makes this concrete: NONE of ludwig2019's four correlations are statistically significant (p=0.207, 0.651, 0.736, 0.651) - with only 8 cell types, a Spearman rho anywhere in a wide range around zero is indistinguishable from noise.

A numerical quirk worth documenting: h1_hescs (n=4) shows two PERFECT correlations (rho=1.0 for type 2 diabetes and red blood cell traits). SciPy's spearmanr reports pvalue=0.0 for these, but that is a known breakdown of its t-distribution approximation at |rho|=1 (division by zero), not the correct value. Brute-force enumeration of all 4! = 24 rank permutations for n=4 gives the true exact two-sided p-value as 2/24 = 0.083 -

not significant at the conventional 0.05 threshold, despite being the best possible outcome achievable with only 4 data points. No individual rho from a collection with n<12 or so should be read as evidence of agreement or disagreement on its own; the larger collections (n=15-175) carry the real statistical weight in Section 5.2.

5.5 Case study: calderon’s near-zero correlation (median rho=0.122)

Unlike the small-collection cases above, calderon (n=175, among the largest collections) has a robust, well-powered near-zero correlation for 2 of its 4 traits that is not a statistical-power artifact.

Structure: calderon’s 175 bigWigs are not 175 distinct biological cell types - they are 45 distinct cell-type/stimulation conditions, each replicated across ~4 donors (e.g. 1001-Th1_precursors-S, 1002-Th1_precursors-S, 1004-..., 1008-...). This is the Soskic et al. (2019) T-cell dataset CHEERS was originally validated on (the same 45 condition names ship as test_input/ elsewhere in this repository, donor-pooled); CREST-GV’s collection instead keeps every donor’s bigWig and peak calls unaggregated.

Both methods are substantially donor-noise-dominated in this collection (variance decomposition: within-base-cell-type / total, across the 4 donor replicates of each of the 45 conditions):

Trait	CHEERS donor-noise var.	CREST-GV donor-noise var.	Observed rho
Type 1 diabetes	13.2%	39.4%	0.673
Heart rate	39.1%	49.9%	0.206
Type 2 diabetes	43.4%	57.4%	0.036
Red blood cell traits	45.1%	61.7%	0.037

Correlating two independently noisy measurements is mathematically attenuated by the product of their noise fractions (observable-correlation ceiling approximately sqrt((1-w_cheers)*(1-w_crestgv))). Donor noise alone predicts a ceiling of about 0.73 for type 1 diabetes (observed: 0.673 - right at the ceiling), but only about 0.49 and 0.46 for type 2 diabetes and red blood cell traits - yet their observed rho (0.036, 0.037) sits far BELOW even that reduced ceiling. Donor noise alone does not fully explain those two.

The remaining gap has a biological explanation. calderon is a T-cell/PBMC collection. Type 1 diabetes is T-cell-mediated autoimmune disease - a direct biological match, so both methods detect real, concordant signal despite the noise. Type 2 diabetes (pancreas/liver/muscle/fat) and red blood cell traits (erythroid lineage) are not primarily mediated through T cells at all - there is effectively no true enrichment signal for either method to find in a T-cell-only collection for those two traits. Under a near-null signal, two different statistical tests’ rankings are expected to show close to zero correlation purely by chance, exactly as observed.

Net conclusion: calderon’s low correlation is not a flaw in either tool. It reflects (1) donor-replicate noise diluting an otherwise-real signal for type 1 diabetes, compounded by (2) a genuine absence of true signal, for either method, on traits this T-cell collection was never going to be informative for. (A direct confirmatory re-run on donor-pooled calderon counts was considered but judged unnecessary to support this conclusion, which is already supported by the variance decomposition and the trait-specific biological pattern.)

5.6 Interactive visualization

A per-collection, per-trait scatter-plot comparison (CHEERS vs CREST-GV, toggleable between Z-score and raw p-value views, colored by significance status in each tool) is available at CREST-GV_vs_CHEERS/docs/cheers_vs_crestgv_scatter.html (self-contained,

opens in any browser, no server required). Each subplot's axes are scaled independently, since CHEERS's Z-score is unbounded (reaching into the tens or hundreds for very strong signals, e.g. Z~73 for erythroblasts/red blood cell traits) while CREST-GV's norm statistic is standardized to roughly [-3, 3] by construction - panels should be read for rank/pattern, not absolute axis position.

6. Conceptual differences between the two methods

Reading CREST-GV's own implementation (crestgv/crestgv.py) clarifies that CHEERS and CREST-GV are not two statistics answering the same question - this matters for interpreting both the agreement and the disagreement above.

- CHEERS asks whether a GWAS SNP sits in chromatin that is specifically/disproportionately accessible in one cell type relative to all others in the collection. It requires one shared consensus peak set across all cell types (built by this pipeline, since CREST-GV's collections provide independently-called peaks per cell type) and ranks peaks by a continuous, Euclidean-normalized specificity score.
- CREST-GV asks whether a cell type's overall accessible genome is enriched for trait SNPs relative to a genome-wide background rate (the fraction of 1000 Genomes common variants falling in that cell type's own peaks). It is a binary peak-overlap proportion Z-test, computed independently per cell type against that cell type's own native peak calls - it does not use bigWig signal strength in the statistic at all, and carries no relative-specificity weighting between cell types.

Two consequences follow. First, a locus accessible in every cell type contributes equally to every cell type's CREST-GV statistic, but is down-weighted by CHEERS's specificity ranking - so some disagreement reflects this genuine conceptual difference, not error in either. Second, because CHEERS requires a shared peak space, this pipeline's consensus-merge step means CHEERS and CREST-GV are not even evaluated against literally identical peak coordinates within a 'matched' collection - a further expected source of difference, independent of the statistical model.

A review of CREST-GV's source code surfaced several hardening opportunities (a stale CLI/class parameter mismatch, division-by-zero risks in SE and Z_norm analogous to the bigWigAverageOverBed failure mode found in Section 5.1, and the exact-binomial pmf/sf columns underflowing to exactly 0.0 for strong signals) - tracked separately in CREST-GV/KNOWN_ISSUES.md and orthogonal to the comparison results above, which use CREST-GV's already-computed, supported output columns (CREST-GV_pval, CREST-GV_pval_norm).

7. Conclusion

CHEERS and CREST-GV show positive, generally moderate-to-strong rank concordance (median rho=0.594 across 48 well-defined collection/trait pairs) on cell types they both identify as most relevant to a trait, and both independently recover biologically expected top hits per trait (Section 5.3). Where they diverge strongly, the divergence is explainable: either the collection is too small to measure concordance reliably at all (Section 5.4), or - as in calderon - a combination of unaggregated biological-replicate noise and genuine absence of trait-relevant signal in that specific reference collection (Section 5.5). Neither pattern indicates a defect in either method; together with the conceptual differences in Section 6, they explain the full range of agreement and disagreement observed.